

EXPERIMENT 01: MICROSOFT AZURE

AIM:

To study Microsoft Azure and its cloud services.

WHAT IS AZURE?

- Microsoft Azure is a cloud computing platform provided by Microsoft.
- It helps individuals and organizations build, deploy, and manage applications.
- It allows storing data and accessing services without maintaining physical servers.
- Azure provides services like virtual machines, databases, networking, and web hosting.

WHY AZURE IS NEEDED?

- Provides high availability and reliability of applications.
- Allows easy scalability based on user demand.
- Reduces the need for costly physical infrastructure.
- Supports global access and remote working.

KEY FEATURES OF AZURE:

- Supports high availability and automatic scaling.
- Provides load balancing and traffic management.
- Offers flexible cloud models like:
 - Infrastructure as a Service (IaaS)
 - Platform as a Service (PaaS)
 - Software as a Service (SaaS)
- Supports hybrid cloud (on-premises + cloud integration).

WEBSITE SERVICES IN AZURE:

- Azure Static Web Apps
- Azure App Services
- Azure Cosmos DB
- Azure DevOps
- Azure Container Apps
- Azure Functions

CLOUD SERVICE MODELS (IaaS, PaaS, SaaS):

1. Infrastructure as a Service (IaaS):

- Provides basic computing resources like virtual machines, storage, and networking.
- Users manage operating systems, applications, and data.
- Example: Azure Virtual Machines

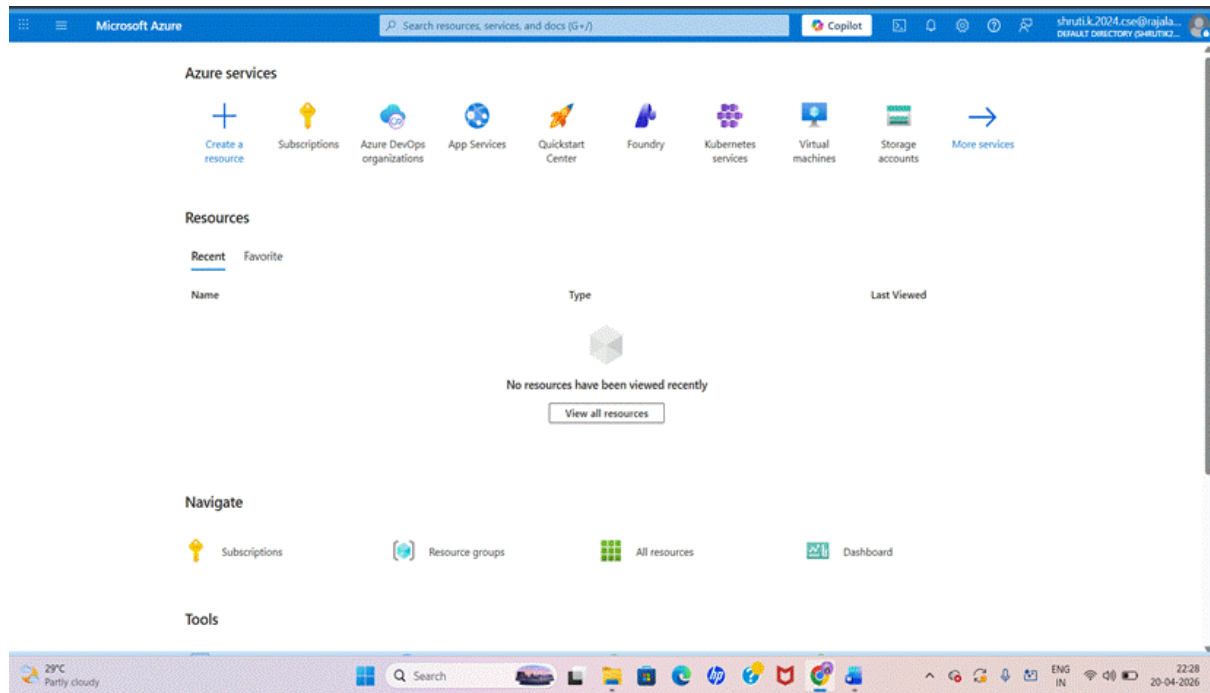
2. Platform as a Service (PaaS):

- Provides a platform to develop, run, and deploy applications.
- Users manage applications and data, while Azure manages OS and infrastructure.
- Example: Azure App Services

3. Software as a Service (SaaS):

- Provides fully developed software applications over the internet.
- Users only use the application; everything else is managed by the provider.
- Example: Microsoft 365

Feature	IaaS	PaaS	SaaS
Control Level	High	Medium	low
User Manages	OS, Apps, Data	Apps, Data	Only usage
Provider Manages	Hardware	OS + Hardware	everything
Flexibility	Very High	Moderate	low
Example	Azure Virtual Machines	Azure App Services	Microsoft 365



ADVANTAGES OF AZURE:

- Cost-effective (pay-as-you-use model)
- Secure and reliable
- Easy deployment and management
- Supports multiple programming languages
- Integration with Microsoft tools

RESULT:

The study of Microsoft Azure was successfully completed.

EXPERIMENT 02: PROBLEM STATEMENT

AIM:

The aim of this experiment is to study the challenges faced in ensuring women's safety in public and private spaces and to design a smart, user-centric system that enables incident reporting, emergency response, and real-time safety monitoring.

PROBLEM STATEMENT:

In today's society, women often face safety concerns such as harassment, unsafe environments, delayed emergency response, and lack of immediate support during critical situations. Although various safety measures exist, they are mostly fragmented and not easily accessible in real-time. Women rely on multiple disconnected sources such as helplines, personal contacts, or local authorities during emergencies. However, the main issue is not the absence of safety mechanisms, but the lack of a unified and intelligent system that can provide immediate assistance, real-time tracking, and proactive safety alerts.

Additionally, many incidents go unreported due to fear, lack of anonymity, or complicated reporting procedures. Even when reports are submitted, there is often no proper analysis of patterns or identification of high-risk areas, which limits preventive action. This problem highlights the need for a centralized, AI-powered safety platform that allows users to report incidents easily, trigger emergency alerts, share live location, and receive support instantly. The system should also assist authorities by providing insights into incident patterns and risk zones. By addressing this issue, the system can enhance women's safety, enable faster emergency response, increase awareness of unsafe areas, and create a more secure environment.

OBJECTIVES:

- To identify challenges related to women's safety and emergency response
- To analyze limitations of existing safety systems and reporting methods
- To design a centralized and user-friendly safety platform
- To enable real-time incident reporting and emergency alert mechanisms
- To integrate AI for risk analysis and prediction of unsafe zones
- To improve communication between users, emergency contacts, and authorities

PROPOSED SOLUTION

- Develop a web-based platform (S.P.E.A.K) for women safety
- Provide an easy incident reporting system with evidence upload
- Implement SOS emergency feature with instant alert triggering
- Integrate emergency contacts for direct calling support
- Enable live location tracking during emergencies
- Use AI module to analyze incidents and predict high-risk zones
- Display incidents on a real-time safety map
- Send notifications and alerts to users and administrators
- Ensure privacy with anonymous reporting options

OUTCOME:

The system provides a centralized platform where users can report incidents, trigger emergency alerts, and receive real-time assistance. It improves safety awareness, enables faster response during emergencies, and helps in identifying high-risk areas through data analysis.

RESULT:

The study highlights the need for an intelligent, centralized women safety system that integrates reporting, emergency response, and AI-based analysis. The proposed solution improves personal safety, enhances emergency communication, and supports authorities in monitoring and preventing unsafe situations, thereby creating a safer and more responsive environment.

EXPERIMENT 03: USE CASE DIAGRAM DESIGN

1. Aim

To design and understand a UML Use Case Diagram representing the interaction between different actors and the SPEAK system for report submission, severity analysis, and emergency alert handling.

2. Objective

- To illustrate the functional requirements of the system
- To identify the actors interacting with the system
- To understand how users and admins interact with the platform
- To represent emergency alert handling for critical situations
- To visualize the relationship between report submission, severity mapping, and higher authority notification
- To study system behavior for both normal and high-risk cases

3. Definition of Use Case Diagram

A Use Case Diagram is a type of UML (Unified Modeling Language) diagram that represents the functional behavior of a system from the user's perspective.

It shows:

- Different users (actors) interacting with the system
- Functionalities provided by the system
- Relationships between actors and use cases
- Include and extend relationships between system functions
- Overall system interaction in a simple visual form

Use Case Diagrams are widely used to represent system requirements and user interactions.

4. Procedure

Step 1: Identify Actors

List all participating actors:

- User
- Admin
- Higher Authority

Step 2: Define the System

The main system considered:

SPEAK – Secure Platform for Evidence Analysis and Knowledge

The system allows users to report incidents, upload evidence, and helps admins analyze severity and manage emergency alerts.

Step 3: Draw System Boundary

- Draw one large rectangle
- Label it as **SPEAK System**

This represents the complete system boundary.

Step 4: Add Use Cases

Inside the system boundary, add oval-shaped use cases:

- Login / Registration
- Report Incident
- Upload Evidence
- View Report Status
- Evidence Verification
- Severity Analysis
- Severity Mapping
- Emergency Alert
- Case Management
- Dashboard Monitoring
- Notifications
- Send to Higher Authority
- View Critical Reports

Step 5: Connect Actors to Use Cases

User connects to:

- Login / Registration
- Report Incident
- Upload Evidence
- View Report Status
- Notifications

Admin connects to:

- Login / Registration
- Evidence Verification
- Severity Analysis
- Severity Mapping
- Emergency Alert
- Case Management
- Dashboard Monitoring

Higher Authority connects to:

- Emergency Alert
- View Critical Reports

Step 6: Add Include Relationships

Use <<include>> relationships for:

- Report Incident → Upload Evidence
- Severity Analysis → Severity Mapping
- Emergency Alert → Send to Higher Authority

Step 7: Finalize Diagram

- Use proper stick figures for actors
- Use oval shapes for use cases
- Use straight connectors
- Maintain clear spacing and neat alignment
- Ensure proper UML structure and readability

5. Notations Used in the Diagram

Notation	Meaning
Stick Figure	Represents actors like User, Admin
Rectangle Box	Represents system boundary
Oval Shape	Represents system functionality
Solid Line	Association between actor and use case
<<include>>	Mandatory included functionality
Labels	Describe use case names

6. Procedure to Create Use Case Diagram in Draw.io

Step 1: Open Draw.io

- Open Draw.io
- Select a location to save your diagram

Step 2: Create a New Diagram

- Click on “Create New Diagram”
- Choose Blank Diagram
- Give a name (e.g., Use Case Diagram)
- Click Create

Step 3: Enable UML Shapes

- On the left panel, click “More Shapes”
- Tick UML → UML Use Case
- Click Apply

Step 4: Add Actors

- Drag Actor (stick figure) shapes
- Add:
 - User
 - Admin
 - Higher Authority

Step 5: Draw System Boundary

- Drag a rectangle shape
- Label it as **SPEAK System**

Step 6: Add Use Cases

- Drag oval use case shapes
- Add all required functionalities inside the system

Step 7: Add Connections

- Use connector lines to connect actors to use cases
- Add include relationships using dashed arrows

Step 8: Final Formatting

- Align all components properly
- Use consistent spacing
- Add clear labels
- Keep the diagram neat and professional

7. Procedure to Upload Diagram in Azure DevOps Wiki

Step 1: Open Azure DevOps

- Go to your project in Azure DevOps
- Navigate to Wiki section

Step 2: Open/Create Page

- Click on your wiki
- Select existing page or click New Page
- Give title (e.g., Use Case Diagram)

Step 3: Enter Edit Mode

- Click Edit button

Step 4: Upload Image

- Drag and drop your diagram directly into editor

OR

- Click Insert Image option
- Upload from your system

Step 5: Adjust Image

- Resize if needed
- Add caption if required

Step 6: Save Page

- Click Save
- Add commit message

Step 7: Verify Upload

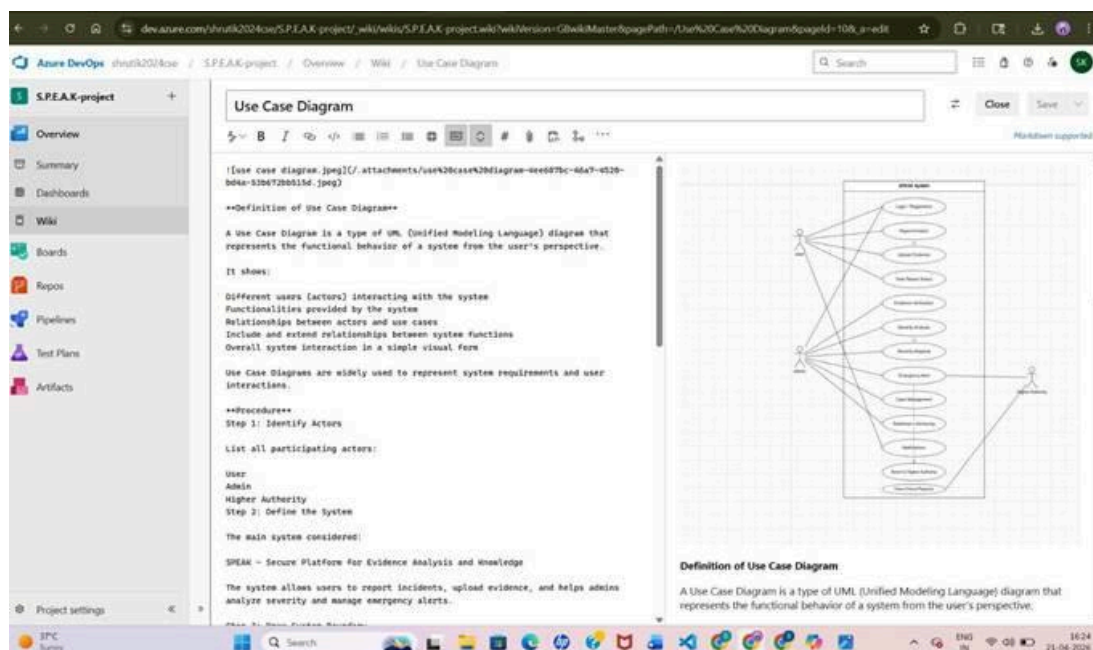
- Check if diagram is visible properly
- Ensure clarity and readability

OUTCOME:

The Use Case Diagram was successfully created to represent the interaction between users, admins, and higher authorities in the SPEAK system.

It clearly illustrates report submission, evidence upload, severity analysis, emergency alert generation, and higher authority notification.

The diagram effectively demonstrates how the system handles both normal and critical situations using proper access control and workflow management.



8. Result

The Use Case Diagram was successfully designed for the SPEAK project. It clearly represents system functionalities, actor interactions, and emergency handling processes. The diagram helps in understanding the complete workflow of the system and supports better system analysis and project presentation.

EXPERIMENT 04: ENTITY RELATIONSHIP (ER) DIAGRAM DESIGN

1. Aim

To design and understand an Entity Relationship (ER) diagram representing the database structure of a safety monitoring system.

2. Objective

- To identify entities involved in the system
- To define attributes of each entity
- To establish relationships between entities
- To understand primary keys and foreign keys
- To model real-world data into a structured database design
- To represent constraints and dependencies between entities

3. Definition of ER Diagram

An Entity Relationship (ER) diagram is a type of diagram used to represent the logical structure of a database.

It shows:

- Entities (tables)
- Attributes (fields/columns)
- Relationships between entities
- Primary Keys (PK) and Foreign Keys (FK)
- Cardinality (one-to-one, one-to-many relationships)

ER diagrams are widely used in database design to visualize how data is stored and connected..

4. Procedure

Step 1: Identify Entities

From the system, the following entities are identified:

- USER
- ADMIN
- REPORT

- SOS_ALERT
- EVIDENCE
- AI_ANALYSIS
- NOTIFICATION

Step 2: Define Attributes

USER

- user_id (PK)
- name
- email
- location
- is_anonymous

ADMIN

- admin_id (PK)
- name
- role
- access_level

REPORT

- report_id (PK)
- description
- location
- severity
- status
- timestamp
- user_id (FK)
- admin_id (FK)

SOS_ALERT

- alert_id (PK)
- location
- time
- status
- user_id (FK)
- admin_id (FK)

EVIDENCE

- evidence_id (PK)
- file_path
- file_type
- upload_time
- report_id (FK)

AI_ANALYSIS

- analysis_id (PK)
- risk_level
- confidence_score
- detected_pattern
- report_id (FK)

NOTIFICATION

- notification_id (PK)
- type
- message
- status
- alert_id (FK)

Step 3: Identify Relationships

- **USER → REPORT**
A user submits reports
- **ADMIN → REPORT**
Admin manages reports
- **USER → SOS_ALERT**
User triggers SOS alerts
- **ADMIN → SOS_ALERT**
Admin handles SOS alerts
- **REPORT → EVIDENCE**
A report contains evidence files
- **REPORT → AI_ANALYSIS**
Each report is analyzed by AI
- **SOS_ALERT → NOTIFICATION**
SOS alerts generate notifications

Step 4: Define Cardinality

- One USER → Many REPORTS (1 to many)
- One ADMIN → Many REPORTS (1 to many)
- One USER → Many SOS_ALERTS (1 to many)
- One ADMIN → Many SOS_ALERTS (1 to many)

- One REPORT → Many EVIDENCE (1 to many)
- One REPORT → One AI_ANALYSIS (1 to 1)
- One SOS_ALERT → Many NOTIFICATIONS (1 to many)

Step 5: Assign Keys

- **Primary Keys (PK):**

Uniquely identify each record

Example: user_id, report_id

- **Foreign Keys (FK):**

Establish relationships between tables

Example:

- report.user_id → USER
- report.admin_id → ADMIN
- evidence.report_id → REPORT

Step 6: Draw ER Diagram

- Represent entities as rectangles
- Add attributes inside each entity
- Mark PK and FK clearly
- Connect entities using relationship lines
- Label relationships (submits, manages, triggers, etc.)

Step 7: Finalize Diagram

- Ensure all entities are properly connected
- Maintain clarity and spacing
- Verify keys and relationships
- Check consistency in naming

5. Notations Used in the Diagram

Notation	Meaning
Rectangle	Entity (table)
Attributes	Columns inside entity
PK	Primary Key
FK	Foreign Key
Line	Relationship between entities
Crow's Foot	Represents “many”
Single Line	Represents “one”
Relationship Label	Describes interaction

6. Procedure to Create ER Diagram in Draw.io

Step 1: Open Draw.io

Go to <https://draw.io>
Select storage option

Step 2: Create New Diagram

Click **Create New Diagram**
Choose **Blank Diagram**
Name it (ER Diagram)

Step 3: Enable ER Shapes

Click **More Shapes**
Enable **Entity Relation**
Click Apply

Step 4: Add Entities

Drag entity shapes and create:
USER, ADMIN, REPORT, SOS_ALERT, EVIDENCE, AI_ANALYSIS, NOTIFICATION

Step 5: Add Attributes

Insert all attributes inside entities
Mark PK and FK clearly

Step 6: Draw Relationships

Connect entities using connectors
Label relationships:

- submits
- manages
- triggers
- handles
- contains
- analyzed_by
- generates

Step 7: Add Cardinality

Use crow's foot notation:

- One (|)
- Many (crow's foot)

Step 8: Final Formatting

Align entities neatly

Maintain spacing

Ensure readability

Step 9: Export Diagram

File → Export As → PNG/JPEG/PDF

7. Procedure to Upload Diagram in Azure DevOps Wiki

Step 1: Open Azure DevOps

- Go to your project in **Azure DevOps**
- Navigate to **Wiki** section

Step 2: Open/Create Page

- Click on your wiki (e.g., *S.P.E.A.K-project wiki*)
- Select existing page or click **New Page**
- Give title (e.g., *Class Diagram*)

Step 3: Enter Edit Mode

- Click **Edit** button on top right

Step 4: Upload Image

- Drag and drop your exported diagram directly into editor
- OR**
- Click **Insert Image** option
 - Upload from your system

Step 5: Adjust Image

- Resize if needed
- Add caption (optional)

Step 6: Save Page

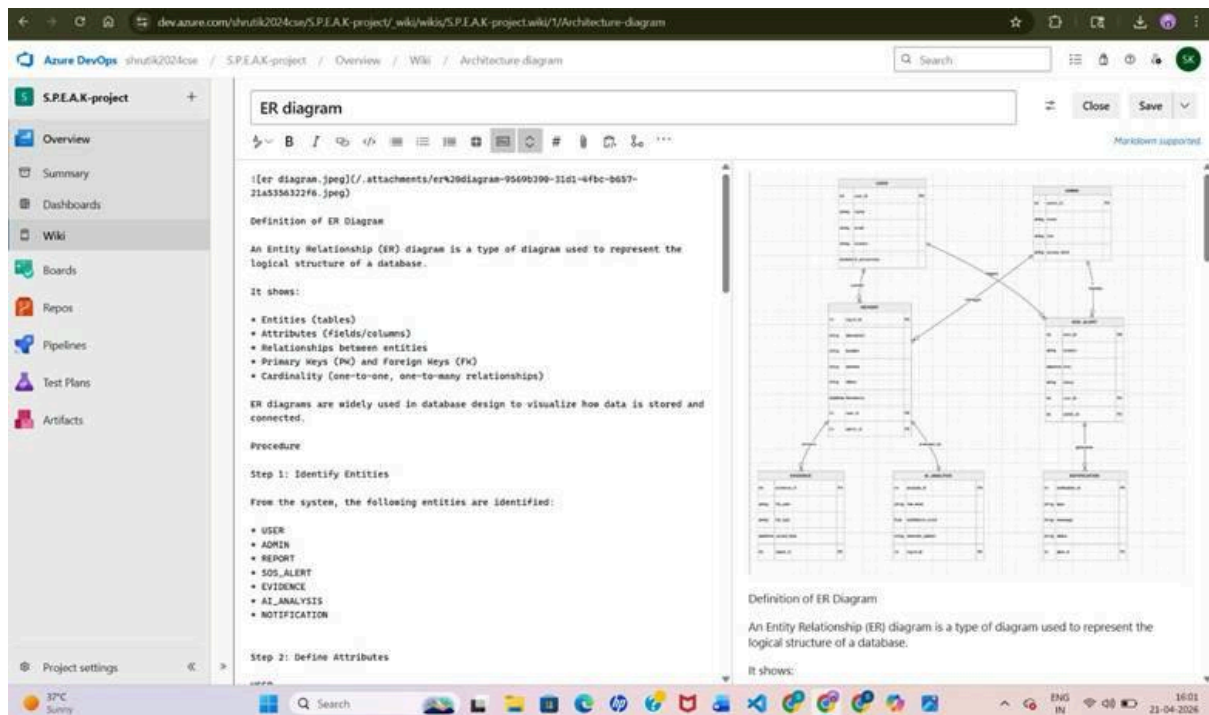
- Click **Save** or **Save & Close**
- Add commit message (e.g., *Added class diagram*)

Step 7: Verify Upload

- Check if diagram is visible properly

- Ensure clarity and readability

7. Outcome



8. Result

The ER diagram was successfully created to represent the database structure of the safety monitoring system. It clearly illustrates entities, attributes, and relationships, including key constraints. The diagram effectively models how data such as reports, alerts, evidence, and notifications are interconnected, ensuring efficient data organization and retrieval.

EXPERIMENT 05: CLASS DIAGRAM DESIGN

1. Aim

To design and understand a class diagram representing the structure of a safety monitoring system, including its classes, attributes, methods, and relationships.

2. Objective

- To identify the main classes in the system
- To define attributes and methods of each class
- To understand relationships between classes
- To represent system structure using UML notation
- To model interactions like report creation, analysis, and emergency handling
- To study how data flows between different system components

3. Definition of Class Diagram

A class diagram is a type of UML (Unified Modeling Language) diagram that represents the static structure of a system.

It shows:

- Classes and their attributes
- Methods/functions of each class
- Relationships between classes (association, composition, etc.)
- Multiplicity (one-to-many, many-to-many relationships)

Class diagrams are widely used for designing object-oriented systems and understanding system architecture.

4. Procedure

Step 1: Identify Classes

From the system, the following classes are identified:

- User
- Admin
- Report
- Evidence
- SOSAlert
- AIAalyzer
- Dashboard

Step 2: Define Attributes and Methods

User

- Attributes: userID, name, location, isAnonymous
- Methods: login(), submitReport(), triggerSOS()

Admin

- Attributes: adminID, name, role
- Methods: verifyReport(), manageReports(), handleSOS()

Report

- Attributes: reportID, description, location, severity, status
- Methods: createReport(), updateStatus()

Evidence

- Attributes: evidenceID, filePath, fileType
- Methods: uploadFile()

SOSAlert

- Attributes: alertID, location, time, status
- Methods: sendAlert()

AIAnalyzer

- Methods: analyzeReport(), classifyRisk()

Dashboard

- Attributes: totalReports, highRiskCount
- Methods: generateStats()

Step 3: Identify Relationships

- **User → Report**
User creates reports (one user can create multiple reports)
- **Admin → Report**
Admin manages reports (one admin can manage many reports)
- **User → SOSAlert**
User triggers SOS alerts
- **Admin → SOSAlert**
Admin handles SOS alerts

- **Report → Evidence**
A report contains multiple evidence files (composition relationship)
- **AIAnalyzer → Report**
AI analyzes reports
- **Dashboard → Report**
Dashboard uses report data for statistics

Step 4: Define Multiplicity

- **One User → Many Reports (1 to 0..*)**
- **One Admin → Many Reports (1 to 0..*)**
- **One Report → Many Evidence (1 to 1..*)**
- **One User → Many SOS Alerts (1 to 0..*)**
- **One Admin → Many SOS Alerts (1 to 0..*)**

Step 5: Add Relationships in Diagram

- Association lines between classes
- Arrows to indicate direction
- Labels such as *creates*, *manages*, *handles*, *triggers*, *analyzes*
- Use diamond symbol for composition (Report → Evidence)

Step 6: Finalize Diagram

- Ensure proper alignment of classes
- Clearly separate attributes and methods
- Add visibility symbols (+ for public)
- Maintain readability and spacing

5. Notations Used in the Diagram

Notation	Meaning
Class Box	Represents a class with attributes and methods
+ Symbol	Public visibility

Attributes	Variables of a class
Methods	Functions of a class
Association Line	Relationship between classes
Arrow →	Direction of relationship
Multiplicity (1, 0.., 1..)	Number of instances
Diamond (◆)	Composition relationship
Dashed Arrow	Dependency

6. Procedure to Create Sequence Diagram in Draw.io

Step 1: Open Draw.io

- Go to <https://draw.io> (or diagrams.net)
- Select **Device** or **Google Drive** to save your file

Step 2: Create a New Diagram

- Click on **“Create New Diagram”**
- Choose **Blank Diagram**
- Give a name (e.g., *Class Diagram*)
- Click **Create**

Step 3: Enable UML Shapes

- On the left panel, click **“More Shapes”**
- Tick **UML → UML Sequence**
- Click **Apply**

Step 4: Add Classes

Drag **Class** shapes and create:

User, Admin, Report, Evidence, SOSAlert, AIAalyzer, Dashboard

Step 5: Add Attributes & Methods

Divide each class into 3 sections:

- Class Name
- Attributes
- Methods

Enter details accordingly

Step 6: Draw Relationships

Use connectors to link classes

Label relationships:

- creates
- manages
- triggers
- handles
- analyzes

Step 7: Add Multiplicity

Add values near connectors:

- 1
- 0..*
- 1..*

Step 8: Add Composition

Use diamond symbol between:

Report and Evidence

Step 9: Final Formatting

Align all elements

Maintain equal spacing

Ensure readability

Step 10: Export Diagram

- Go to **File** → **Export As** → **PNG/JPEG/PDF**
- Save the file to your system

7. Procedure to Upload Diagram in Azure DevOps Wiki

Step 1: Open Azure DevOps

- Go to your project in **Azure DevOps**
- Navigate to **Wiki section**

Step 2: Open/Create Page

- Click on your wiki (e.g., *S.P.E.A.K-project wiki*)
- Select existing page or click **New Page**
- Give title (e.g., *Class Diagram*)

Step 3: Enter Edit Mode

- Click **Edit** button on top right

Step 4: Upload Image

- Drag and drop your exported diagram directly into editor
- OR**
- Click **Insert Image** option
 - Upload from your system

Step 5: Adjust Image

- Resize if needed
- Add caption (optional)

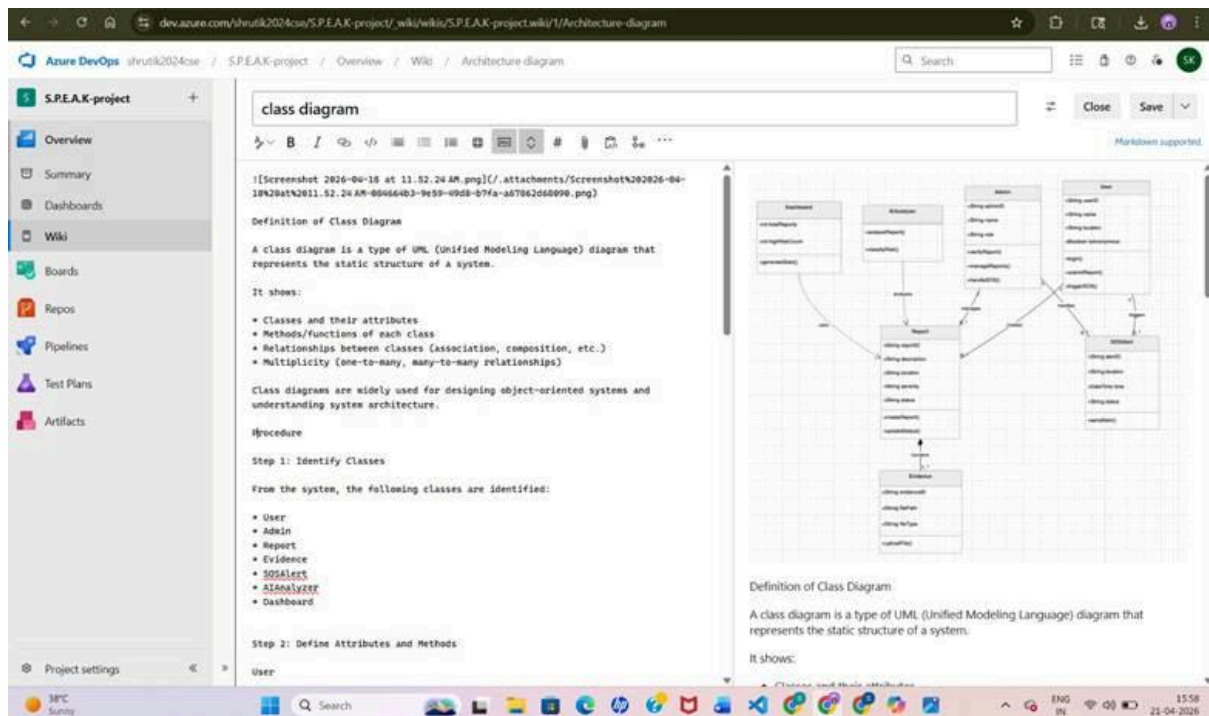
Step 6: Save Page

- Click **Save** or **Save & Close**
- Add commit message (e.g., *Added class diagram*)

Step 7: Verify Upload

- Check if diagram is visible properly
- Ensure clarity and readability

7. Outcome



8. Result

The class diagram was successfully created to represent the static structure of the safety monitoring system. It clearly illustrates the relationships between users, admins, reports, evidence, and alert mechanisms. The diagram effectively models how different components interact structurally, enabling better understanding of system design and object-oriented implementation.

EXPERIMENT 06: ACTIVITY DIAGRAM DESIGN

Aim

To design and understand an activity diagram representing the workflow and control flow of a safety monitoring system including user actions, system processing, AI analysis, and administrative operations.

Objective

- To represent the workflow of the system using activity modeling
- To understand control flow between different activities
- To visualize decision-making and parallel processes
- To model user, system, and admin interactions
- To analyze the flow of actions from start to end

Definition of Activity Diagram

An **Activity Diagram** is a type of UML (Unified Modeling Language) diagram that represents the flow of activities or actions within a system.

It is used to:

- Model workflows and business processes
- Show sequence of activities
- Represent decisions and branching
- Illustrate parallel and conditional flows

Unlike sequence diagrams, activity diagrams focus on **flow of control**, not message exchange.

Procedure

Step 1: Identify Activities

Main activities in the system:

- User login/register
- Report incident
- SOS alert handling
- AI analysis
- Admin verification

Step 2: Define Start and End Nodes

- Start node: User opens application
- End node: Process completion after admin action

Step 3: Model User Activities

- Open application
- Register or login
- Decision: Login successful?
 - If No → Retry
 - If Yes → Access dashboard

Step 4: Report Submission Flow

- Report incident
- Enter details and upload evidence
- System validates credentials
- Store report in database

Step 5: Decision – Emergency SOS

- Decision: Emergency SOS?
 - If Yes:
 - Trigger SOS alert
 - Enable live tracking
 - If No:
 - Continue normal flow

Step 6: System Activities

- Update incident status
- Update safety map
- Process via AI module
- Generate notifications
- Notify admin and emergency contacts

Step 7: AI Processing Flow

- Analyze report
- Predict risk and patterns
- Decision: High-risk zone?
 - If Yes → Mark critical and notify admin
 - If No → Suggest preventive actions

Step 8: User Interaction

- View safety map/zones
- Receive alerts and feedback

Step 9: Admin Activities

- Admin login
- View reports and maps
- Categorize and verify reports
- Decision: Action required?
 - If Yes:
 - Initiate response
 - Review AI recommendations
 - Manage users

Step 10: End Flow

- Process ends after completion of actions

Procedure to Create Activity Diagram in Draw.io

Step 1: Open Draw.io

- Go to diagrams.net
- Create a new blank diagram

Step 2: Enable UML Shapes

- Click **More Shapes**
- Select **UML** → **UML Activity**
- Click Apply

Step 3: Add Start Node

- Drag **Initial Node (●)**

Step 4: Add Activities

- Use rounded rectangles
- Add:
 - Open Application
 - Login/Register
 - Report Incident
 - etc.

Step 5: Add Decision Nodes

- Use diamond shape
- Add conditions:
 - Login successful?
 - Emergency SOS?
 - High-risk zone?

Step 6: Add Control Flow

- Connect shapes using arrows
- Label Yes/No branches

Step 7: Add Swimlanes

- Divide diagram into sections:
 - User
 - System
 - Admin

Step 8: Add End Node

- Use final node symbol

Step 9: Format Diagram

- Align properly
- Maintain spacing
- Ensure clarity

Step 10: Export

- File → Export as PNG/JPEG/PDF

Procedure to Upload in Azure DevOps Wiki

Step 1: Open Azure DevOps

- Navigate to your project → Wiki

Step 2: Create/Open Page

- Create page: *Activity Diagram*

Step 3: Click Edit

- Enter editing mode

Step 4: Upload Diagram

- Drag and drop image
- OR
- Use Insert Image

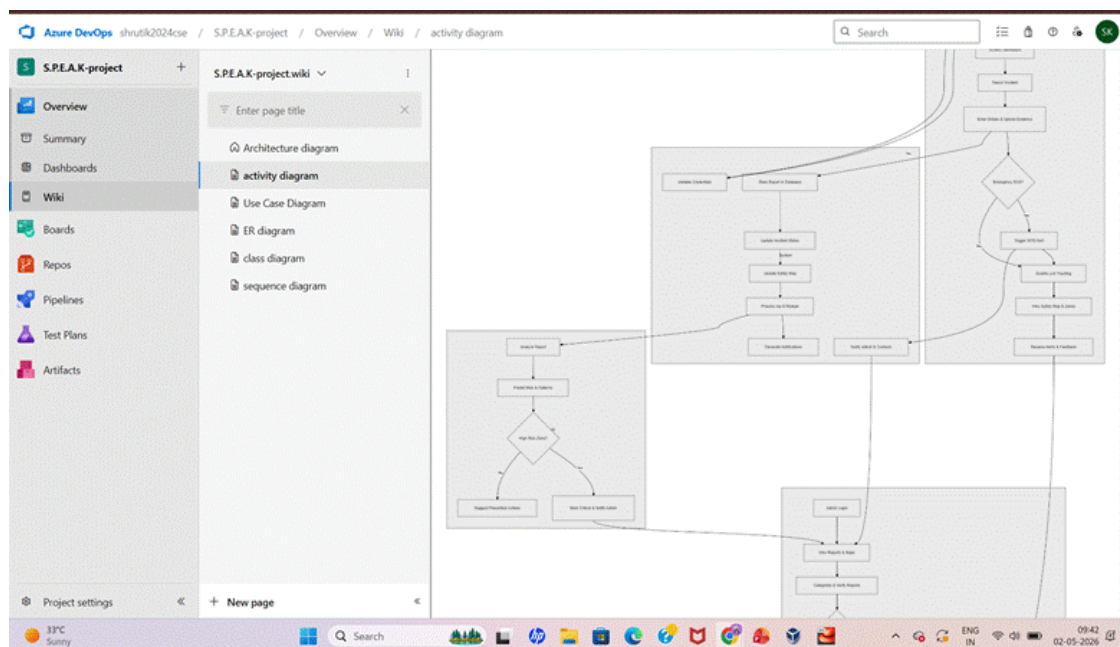
Step 5: Save

- Click Save
- Add commit message

Step 6: Verify

- Ensure diagram is clearly visible

Output screen



Result

The activity diagram was successfully designed, representing the workflow of the system with clear control flow, decision points, and interactions between user, system, and admin. It effectively illustrates the sequence of activities and conditional logic involved in the safety monitoring process.

EXPERIMENT 07: SEQUENCE DIAGRAM DESIGN

1. Aim

To design and understand a sequence diagram representing the interaction between different components of a safety monitoring system during report submission and emergency handling.

2. Objective

- To illustrate the step-by-step flow of events in the system
- To understand how different components interact over time
- To represent communication between user, frontend, backend, AI module, database, and external services
- To visualize decision-making scenarios such as risk classification
- To study system response for normal and high-risk situations

3. Definition of Sequence Diagram

A **sequence diagram** is a type of UML (Unified Modeling Language) diagram that shows how objects or components interact in a particular scenario over time.

It represents:

- The order of execution of processes
- Communication between system components
- Flow of messages from one component to another
- Conditional flows and decision points

Sequence diagrams are widely used to model dynamic behavior of systems.

4. Procedure

Step 1: Identify Actors and Components

List all participating entities:

- User
- Frontend
- Backend
- AI Analyzer
- Database
- Notification Service
- Admin

Step 2: Define the Scenario

The main scenario considered:

- User submits a report
- System processes and analyzes it
- Risk level is determined
- Actions are taken based on severity

Step 3: Draw Lifelines

- Represent each component as a vertical dashed line
- Place them horizontally in order (User → Admin)

Step 4: Add Message Flow

Initial Interaction

- User enters report details and submits
- Frontend sends report data to backend

Backend Processing

- Backend stores initial report in database
- Sends report to AI Analyzer

AI Analysis

- AI processes report
- Returns risk level (Low / Medium / High)

Database Update

- Backend updates report with risk level

Step 5: Add Conditional Flow (alt block)

Case 1: Low/Medium Risk

- Backend sends confirmation message
- Frontend displays “Report submitted successfully”

Case 2: High Risk

- Backend triggers alert
- Notification service sends alert
- Admin receives notification
- Admin acknowledges and takes action
- User receives urgent confirmation (“Authorities notified”)

Step 6: Add Optional Flow (opt block)

- Admin updates report status
- Backend saves updated status in database

Step 7: Finalize Diagram

- Label all messages clearly
- Use arrows to show direction
- Include conditions (alt, opt blocks)
- Ensure proper sequence from top to bottom

5. Notations Used in the Diagram

Notation	Meaning
Actor (Stick figure / Box)	Represents external entity (User, Admin)
Rectangle Box	Represents system components (Frontend, Backend, etc.)
Lifeline (Vertical dashed line)	Represents existence of an object over time
Solid Arrow →	Message or function call
Dashed Arrow →	Response or return message
Activation Bar	Shows active processing time of a component
Alt Block	Represents conditional flow (if/else cases)
Opt Block	Represents optional execution
Message Labels	Describe action (Send report, Notify, Update status)

6. Procedure to Create Sequence Diagram in Draw.io

Step 1: Open Draw.io

- Go to <https://draw.io> (or diagrams.net)
- Select **Device** or **Google Drive** to save your file

Step 2: Create a New Diagram

- Click on “**Create New Diagram**”

- Choose **Blank Diagram**
- Give a name (e.g., *Sequence Diagram*)
- Click **Create**

Step 3: Enable UML Shapes

- On the left panel, click “**More Shapes**”
- Tick **UML → UML Sequence**
- Click **Apply**

Step 4: Add Participants (Lifelines)

- Drag **Lifeline/Participant** shapes
- Add components:
 - User
 - Frontend
 - Backend
 - AI Analyzer
 - Database
 - Notification Service
 - Admin

Step 5: Draw Lifelines

- Ensure each participant has a **vertical dashed line**
- Arrange them horizontally from left to right

Step 6: Add Messages (Arrows)

- Use **connector arrows** to show communication
- Label each arrow:
 - “Enter report details”
 - “Send report data”
 - “Store initial report”
 - “Send for analysis”
 - “Return risk level”

Step 7: Add Activation Bars

- Drag **activation boxes** onto lifelines
- Show processing duration for:
 - Backend
 - AI Analyzer

Step 8: Add Conditional Blocks

- Use **Frame** → **Alt (Alternative)**
- Divide into:
 - Low/Medium Risk
 - High Risk

Add actions inside each section.

Step 9: Add Optional Block

- Use **Frame** → **Opt (Optional)**
- Add:
 - “Update report status”

Step 10: Final Formatting

- Align all components properly
- Use consistent spacing
- Add clear labels

Step 11: Export Diagram

- Go to **File** → **Export As** → **PNG/JPEG/PDF**
- Save the file to your system

7. Procedure to Upload Diagram in Azure DevOps Wiki

Step 1: Open Azure DevOps

- Go to your project in **Azure DevOps**
- Navigate to **Wiki section**

Step 2: Open/Create Page

- Click on your wiki (e.g., *S.P.E.A.K-project wiki*)
- Select existing page or click **New Page**
- Give title (e.g., *Sequence Diagram*)

Step 3: Enter Edit Mode

- Click **Edit** button on top right

Step 4: Upload Image

- Drag and drop your exported diagram directly into editor
OR
- Click **Insert Image** option

The sequence diagram was successfully created to represent the dynamic interaction between system components. It clearly illustrates the flow of report submission, AI-based risk analysis, conditional decision-making, and emergency alert handling. The diagram effectively demonstrates how the system responds differently to normal and high-risk situations, ensuring efficient communication and timely action.

EXPERIMENT 08: SYSTEM ARCHITECTURE DIAGRAM DESIGN

1. Aim

To design and understand a multi-layered system architecture diagram for a safety monitoring application that integrates client interfaces, backend services, security mechanisms, data storage, and external services.

2. Objective

- To represent the overall structure of the system using an architecture diagram
- To understand the interaction between different system layers
- To identify modules such as user services, AI processing, and analytics
- To study secure data handling using encryption and API gateways
- To visualize data flow between components like database, services, and external APIs

3. Definition of Architecture Diagram

An **architecture diagram** is a visual representation of a system that illustrates its structure, components, layers, and interactions. It helps in understanding how different parts of a system communicate and work together.

In software engineering, architecture diagrams are used to:

- Show system design at a high level
- Represent layers such as client, application, and data
- Illustrate data flow and communication paths
- Improve clarity for developers, stakeholders, and designers

4. Procedure

Step 1: Identify System Layers

Divide the system into logical layers:

- Client Layer
- Security Layer
- Application Layer
- Data Layer
- External Services

Step 2: Design Client Layer

Include user-facing components:

- Web/Mobile App (Login, Dashboard, Reports)
- SOS Emergency Button

- Safety Map Interface

Step 3: Add Security Layer

Incorporate security mechanisms:

- Secure API Gateway (SSL/TLS)
- Anonymous Identity Protection
- End-to-End Data Encryption

Step 4: Define Application Layer

Design backend modules:

- User Module (Authentication, Reports, SOS Trigger)
- AI Analysis Module (Risk classification, pattern detection)
- Admin Module (Monitoring, RBAC)
- Analytics & Visualization Engine

Step 5: Design Data Layer

Include storage components:

- Primary Database (Users, Reports, Locations)
- Analytics Storage (Insights, Trends)

Step 6: Integrate External Services

Connect third-party services:

- Map API (Google/OSM)
- Notification Service (SMS, Email, Push)
- Cloud Storage (Images, Evidence files)

Step 7: Establish Data Flow

Draw arrows to represent:

- Report submission → AI analysis
- AI results → analytics engine
- Alerts → notification service
- Data storage → database

Step 8: Finalize Diagram

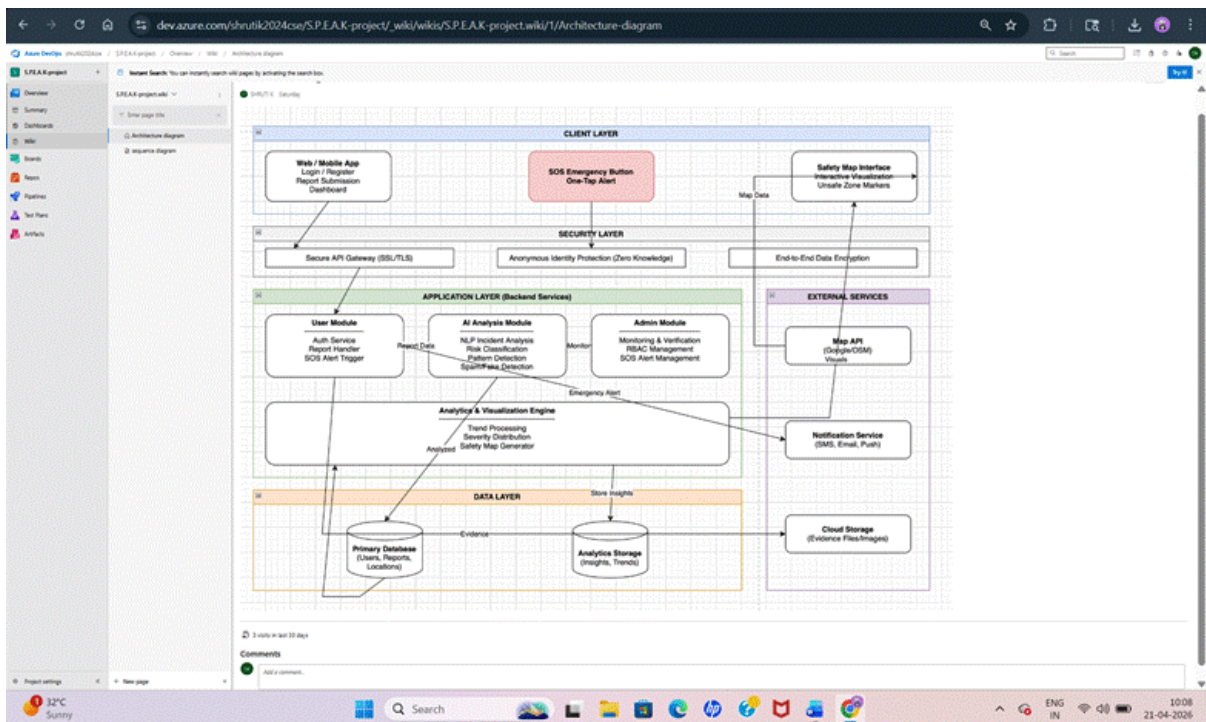
- Label all components clearly
- Use proper shapes and arrows

- Maintain layer-wise organization

5. Notations Used in the Diagram

Notation	Meaning
Rectangle Box	Represents modules/components (User Module, AI Module, etc.)
Layered Sections	Represents different system layers (Client, Security, etc.)
Cylindrical Shape	Represents databases/storage systems
Arrows	Represent data flow or communication between components
Rounded Rectangles	Used for UI elements or services
External Box Section	Represents third-party/external services
Labels on Arrows	Describe type of data (Report Data, Map Data, Alerts)

OUTCOME:



6. Result

The system architecture diagram was successfully designed, illustrating a layered structure that includes client interaction, secure communication, backend processing, data storage, and external integrations. The diagram clearly shows how data flows through the system, how security is maintained, and how different modules collaborate to provide functionalities like incident reporting, AI-based analysis, and emergency alerts.

EXPERIMENT 09: PROJECT IMPLEMENTATION

Aim

To implement the S.P.E.A.K (Secure Platform for Evidence Analysis and Knowledge) system and upload the project files to a version control platform (Azure DevOps/GitHub).

Objective

- To develop the backend modules of the S.P.E.A.K system
- To organize project files in a structured manner
- To upload and manage project code using version control
- To understand repository creation, file upload, and commit tracking
- To maintain project version history for collaboration and future updates

Procedure (Uploading Project to Azure DevOps / GitHub)

Step 1: Prepare Project Folder

- Organize project files into folders:
 - backend
 - frontend
- Ensure all required files (app.py, modules, etc.) are present

Step 2: Initialize Git Repository (Local System)

Step 3: Create Repository (Azure DevOps / GitHub)

- Go to Azure DevOps or GitHub
- Create a new repository
- Copy the repository URL

Step 4: Connect Local Project to Repository

Step 5: Upload Files

- All project files will be uploaded to the repository
- Verify files appear in:
 - backend folder
 - frontend folder

Step 6: Verify Repository

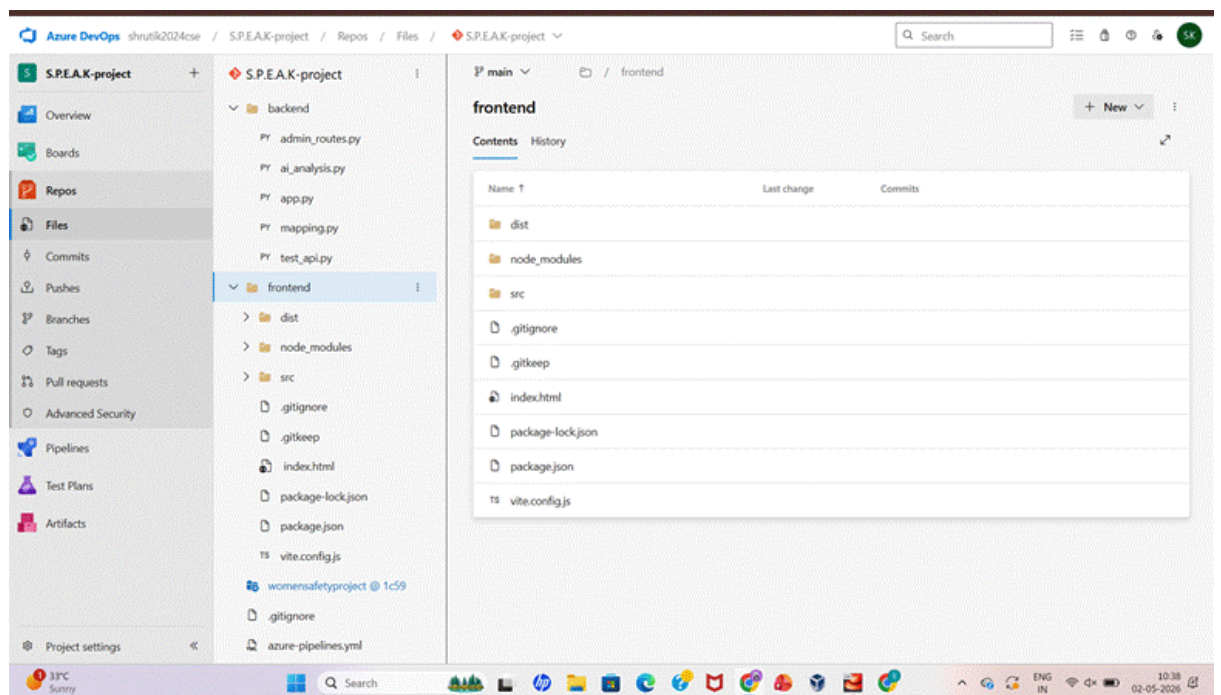
- Open repository in browser
- Check:

- File structure
- Commit history
- Updated changes

GitHub / Azure Repository Link

<https://github.com/Shrutik-24/SPEAK>

Output Screen



Result

The S.P.E.A.K project was successfully implemented and uploaded to a version control platform. All backend modules were properly organized and stored in the repository, ensuring efficient project management and version tracking.

Experiment 10: Azure DevOps Pipeline Execution for S.P.E.A.K

Aim

To create and execute a Continuous Integration (CI) pipeline using Azure DevOps for the S.P.E.A.K project and monitor the pipeline execution process.

Objective

- To understand the working of Azure DevOps Pipelines.
- To automate the build and integration process of the project.
- To monitor pipeline stages and execution status.

Software Requirements

- Azure DevOps Account
- Web Browser (Chrome/Edge)
- Internet Connection
- Git Repository linked with Azure DevOps

Procedure

Step 1: Login to Azure DevOps

Open Azure DevOps using a web browser and sign in with valid credentials.

Step 2: Create/Open Project

Open the project named **S.P.E.A.K-project** from the Azure DevOps dashboard.

Step 3: Navigate to Pipelines

From the left-side menu, select **Pipelines** to access the pipeline management section.

Step 4: Configure CI Pipeline

Create or configure a Continuous Integration pipeline connected to the project repository.

Step 5: Run the Pipeline

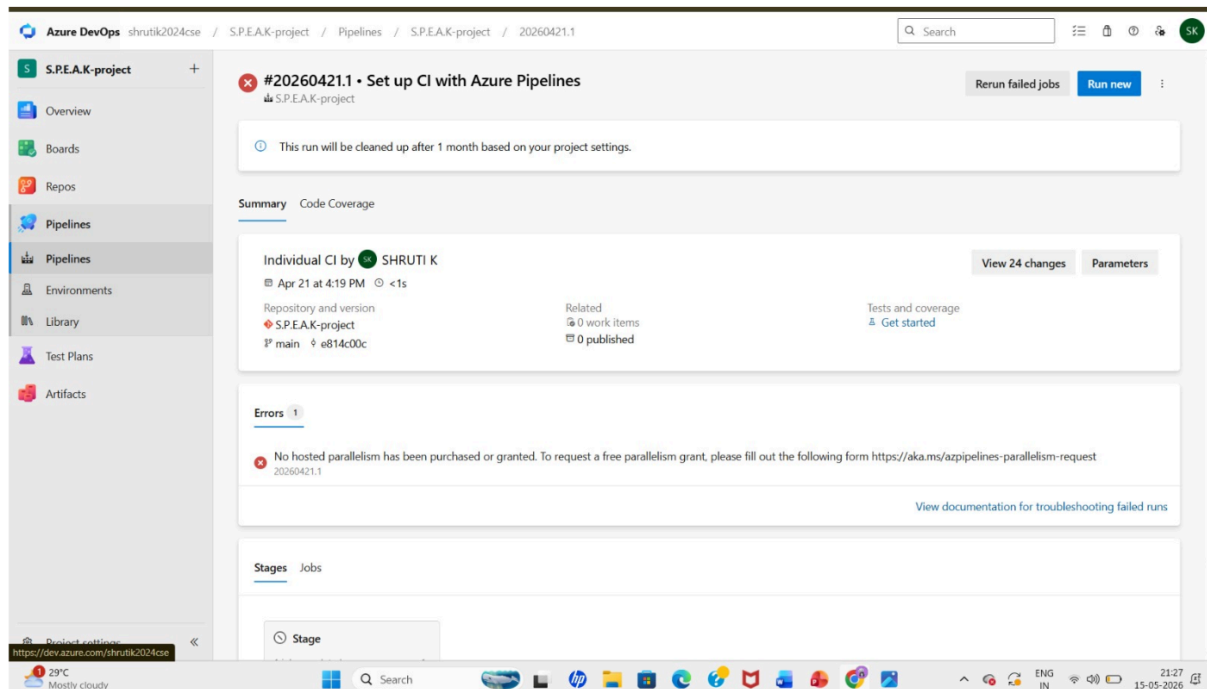
Click the **Run pipeline** button to start the pipeline execution process.

Step 6: Monitor Pipeline Execution

Observe the stages, execution status, branch details, and logs generated during the pipeline run.

Step 7: Verify Output

Check whether the pipeline execution is successful or failed based on the status indicator.



Observation

- The pipeline named **S.P.E.A.K-project** was executed.
- The run ID displayed is **#20260421.1**.
- The pipeline was triggered for the **main** branch.
- The execution status shows a failed stage indicated by the red error symbol.

Result

Thus, the Azure DevOps CI pipeline for the **S.P.E.A.K-project** was successfully created and executed. The pipeline execution status and stages were monitored through the Azure DevOps interface.

EXPERIMENT 11 : TEST CASES

Aim

To test and validate the functionality, security, reliability, and performance of the S.P.E.A.K. women safety and incident reporting system by verifying all modules including report submission, AI analysis, SOS alerts, dashboard analytics, and admin management.

Objectives

- To verify proper functioning of user and admin modules.
- To ensure anonymous incident reporting works correctly.
- To test AI-based risk classification and spam detection.
- To validate SOS emergency alert functionality.
- To ensure secure handling of user data and reports.
- To check dashboard visualization and safety map accuracy.
- To identify and prevent system errors or unauthorized access.

Scope of Testing

The testing process covers:

- User authentication and anonymous access
- Incident report submission
- Evidence upload functionality
- AI-powered analysis and classification
- SOS emergency alert handling
- Dashboard and analytics visualization
- Admin report verification and monitoring
- Database storage and retrieval
- Security and validation testing

Testing Methodology

The following testing methods were used:

Testing Type	Description
Functional Testing	Checks whether all features work correctly

Validation Testing	Verifies correct input handling
Security Testing	Ensures protection from unauthorized access
UI Testing	Tests interface usability and responsiveness
Integration Testing	Verifies communication between modules
System Testing	Ensures complete system functionality

Test Environment

Component	Specification
Frontend	React.js
Backend	Node.js / Express.js
Database	Firebase / MongoDB
Browser	Google Chrome
Operating System	Windows 10 / macOS

Tools Used	VS Code, Postman, Draw.io
------------	---------------------------

Sample Test Cases

Login Module

Test Case ID	Test Scenario	Expected Result	Status
TC_LOGIN_01	Valid Login	User successfully logs in	Pass
TC_LOGIN_02	Invalid Password	Error message displayed	Pass
TC_LOGIN_03	Empty Fields	Validation warning shown	Pass

Report Submission Module

Test Case ID	Test Scenario	Expected Result	Status
TC_REPORT_01	Submit Valid Report	Report stored successfully	Pass
TC_REPORT_02	Missing Description	Validation error displayed	Pass
TC_REPORT_03	Upload Evidence	File uploads successfully	Pass

AI Analysis Module

Test Case ID	Test Scenario	Expected Result	Status
TC_AI_01	High Risk Detection	AI classifies report correctly	Pass
TC_AI_02	Spam Detection	Fake report identified	Pass

SOS Alert Module

Test Case ID	Test Scenario	Expected Result	Status
TC_SOS_01	Trigger SOS Alert	Emergency notification sent	Pass
TC_SOS_02	SOS with GPS	Accurate location shared	Pass

Result

All major modules of the S.P.E.A.K. system were tested successfully.

The application correctly handled report submissions, AI-based analysis, SOS emergency alerts, dashboard visualization, and admin monitoring functionalities. Security checks and validation tests also passed successfully without major issues.

Outcome

- The system provides secure and anonymous reporting.
- AI successfully classifies incident severity levels.
- SOS alerts improve emergency response handling.
- Dashboard analytics help identify unsafe zones and trends.
- The platform ensures better monitoring and women safety awareness.
- The system is reliable, user-friendly, and suitable for real-world implementation.

Conclusion

Testing confirmed that the S.P.E.A.K. platform functions effectively across all major modules. The system achieved its objectives of secure reporting, intelligent analysis, quick emergency support, and administrative monitoring. Proper testing improved system reliability, usability, and overall safety management capabilities.